

8.d. Multi – level Linked List

–Destroy List(Main List and Child)

–Time Complexity Calculation using Masters Theorem

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}
```

Here ,master's theorem only applicable to

Best Structure [For Balanced Tree]

$$2T\left(\frac{N-1}{2}\right) + C, \text{ Base Case: } T(0) = 1.$$

$$2T\left(\frac{N-1}{2}\right) + C \text{ can be approximated as } \approx 2T\left(\frac{N}{2}\right) + C$$

Method 1: The Simplified Master Theorem

This method relies on matching the equation to the specific polynomial form:

$$T(n) = aT\left(\frac{n}{b}\right) + cn^k$$

Step1 : Extract the constants:

$a = 2$ (The number of subproblems)

$b = 2$ (The division factor)

$c = C$ (The constant work done)

$k = 0$ (Because the constant work C doesnot scale with N , it can be written as $C \cdot N^0$)

$$2T\left(\frac{N}{2}\right) + C \cdot N^0$$

Step 2: Evaluate the Condition

The Simplified Master Theorem ask us to compare the constants ' a ' and ' b^k ' to determine which case applies.

- **$a = 2$**
- **$b^k = 2^0 = 1$**

Because $2 > 1$, we clearly see that $a > b^k$.

Step 3: Apply the correct case

- **Case 1 requires $a < b^k$.**
- **Case 2 requires $a = b^k$.**
- **Case 3 requires $a > b^k$.**

Our equation triggers Case 3 , which states that the time complexity is:

$$\Theta(n^{\log_b a})$$

Step 4: The Final Calculation

Substituting the values of a and b into the Case 3 solution:

$$\Theta(n^{\log_2 2})$$

$$= \Theta(n^1)$$

Final Answer: $\Theta(N)$

Method 2: Continuous Generalized Master Theorem

$$T(n) = aT\left(\frac{n}{b}\right) + c \times f(n)$$

$$\text{or, } 2T\left(\frac{N}{2}\right) + C \times N^0$$

Step 1 : Extract the variables:

$$a = 2$$

$$b = 2$$

$$f(N) = N^0$$

Step 2 Calculate $n^{\log_b a}$:

$$\therefore N^{\log_b a} = N^{\log_2 2} = N^1 = N$$

Step 3: Determine ε :

- *Exponent of $N^{\log_b a}$ is 1.*
- *Exponent of $f(n) = N^0 = 0$.*
- *Difference (ε) is : $1 - 0 = 1$.*
- *Therefore $\varepsilon = 1$.*

Step 4: Determine Cases:

1. If there exists a constant $\varepsilon > 0$ such that

$$f(n) = O(n^{\log_b a - \varepsilon}) \Rightarrow n^{\log_b a - \varepsilon} < n^{\log_b a} \text{ then } T(n) = \Theta(n^{\log_b a}).$$

2. If there exists a constant $k \geq 0$ such that

$$f(n) = \Theta(n^{\log_b a} \log^k n), \text{ then } T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$

3. If there exists a constant $\varepsilon > 0$ such that

$$f(n) = \Omega(n^{\log_b a + \varepsilon}), \text{ and if } f(n) \text{ additionally}$$

satisfies the regularity condition

$$af\left(\frac{n}{b}\right) \leq cf(n) \text{ for some constant } c < 1 \text{ and}$$

all sufficiently large n , then $T(n) = \Theta(f(n))$.

We got $\varepsilon > 0$ i.e. $\varepsilon = 1$, for Case 2 it will be required:

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

$$\text{even if } k = 0, f(n) = \Theta(n^{\log_2 2} \log^0 n) \Rightarrow \Theta(n^{\log_2 2} \times (\log n)^0) = \Theta(n^{\log_2 2} \times 1) \\ = \Theta(n) [\log_a a = 1]$$

But our $f(n) = C \times N^0$ or, $\Theta(1)$.

$$f(n) = \Theta(n) \neq f(n) = \Theta(1)$$

not satisfied.

We got $\varepsilon > 0$ i.e. $\varepsilon = 1$, for Case 3 it will be required:

$$f(n) = \Omega(n^{\log_b a + \varepsilon}) = \Omega(n^{\log_2 2 + 1}) = \Omega(n^{\log_2 2 + 1}) = \Omega(n^{1+1}) \\ = \Omega(n^{1+1}) = \Omega(n^2)$$

$$\text{i.e. } F(n) = \Omega(n^{1+1}).$$

$$\Omega(n^{1+\varepsilon}) \neq F(n) = \Theta(1)$$

Hence Case 3, also not possible.

As $\varepsilon > 0$, we proceed with case 1 :

$$f(n) = O(n^{\log_b a - \varepsilon}) = O\left(\frac{n^{\log_b a}}{n^\varepsilon}\right) = O\left(\frac{n^{\log_b a}}{n^\varepsilon}\right) = O\left(\frac{n^{\log_2 2}}{n^1}\right)$$

$$= O\left(\frac{n^{\log_2 2}}{n^1}\right) = O\left(\frac{n^1}{n^1}\right) = O(n^{1-1}) = O(n^0) = O(1).$$

$$\text{As, } n^0 \text{ or } n^{1-\varepsilon} < n^1 \implies n^{\log_b a - \varepsilon} < n^{\log_b a}$$

Therefore, it satisfies Case 1,

$$\text{Hence } T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n^1) = \Theta(n)$$
